

Team Members

Name: Ahmed Ebrahim Hamdy

- **ID:** 23102354
- **Email:** Ahmed.abdelzaher.2024@aiu.edu.eg

2. **Name:** Mahmoud Emad Awad

- **ID:** 23101474
- **Email:** mahmoud.emad.2024@aiu.edu.eg

3. **Name:** Muhammed Hany Hamdy

- **ID:** 23102359
- **Email:** Mohamed.abuaaliyah.2024@aiu.edu.eg

4. **Name:** Mostafa Khamis Abozead

- **ID:** 23101484
- **Email:** Mostafa Khamis Abozead

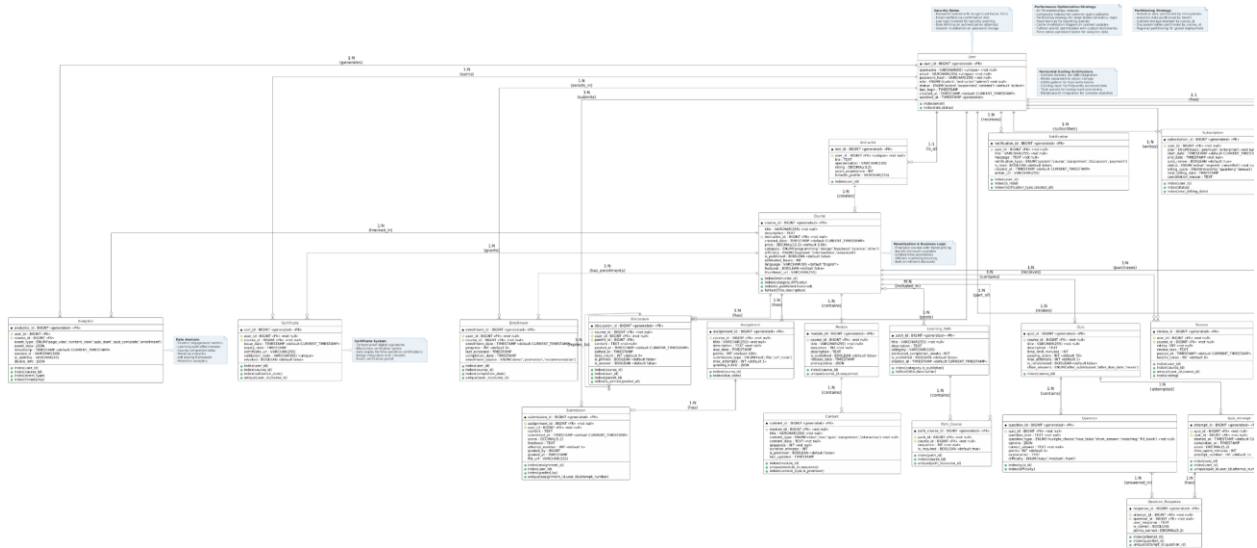
5. **Name:** Mohamed Soliman Habib

- **ID:** 23101462
- **Email:** mohamed.abdelhamid.2024@aiu.edu.eg

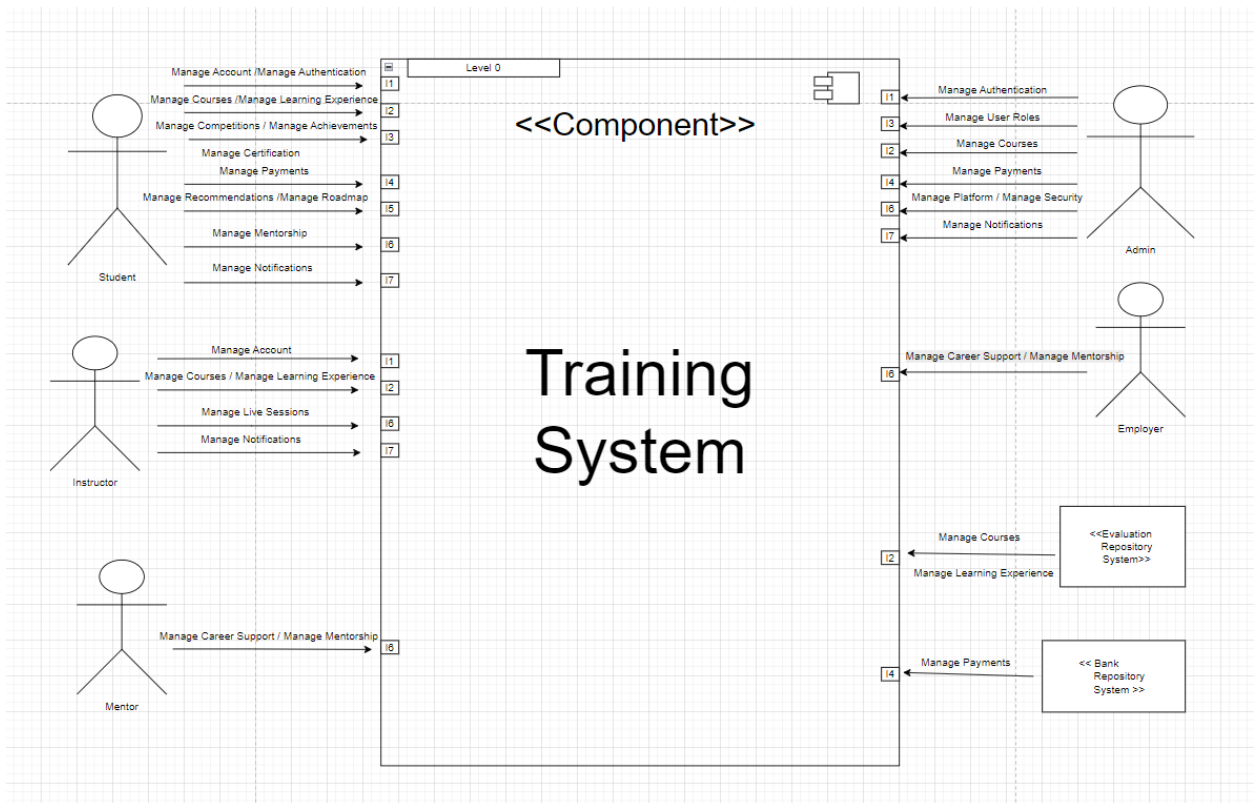
Database ERD

[Link for Photo:](#)

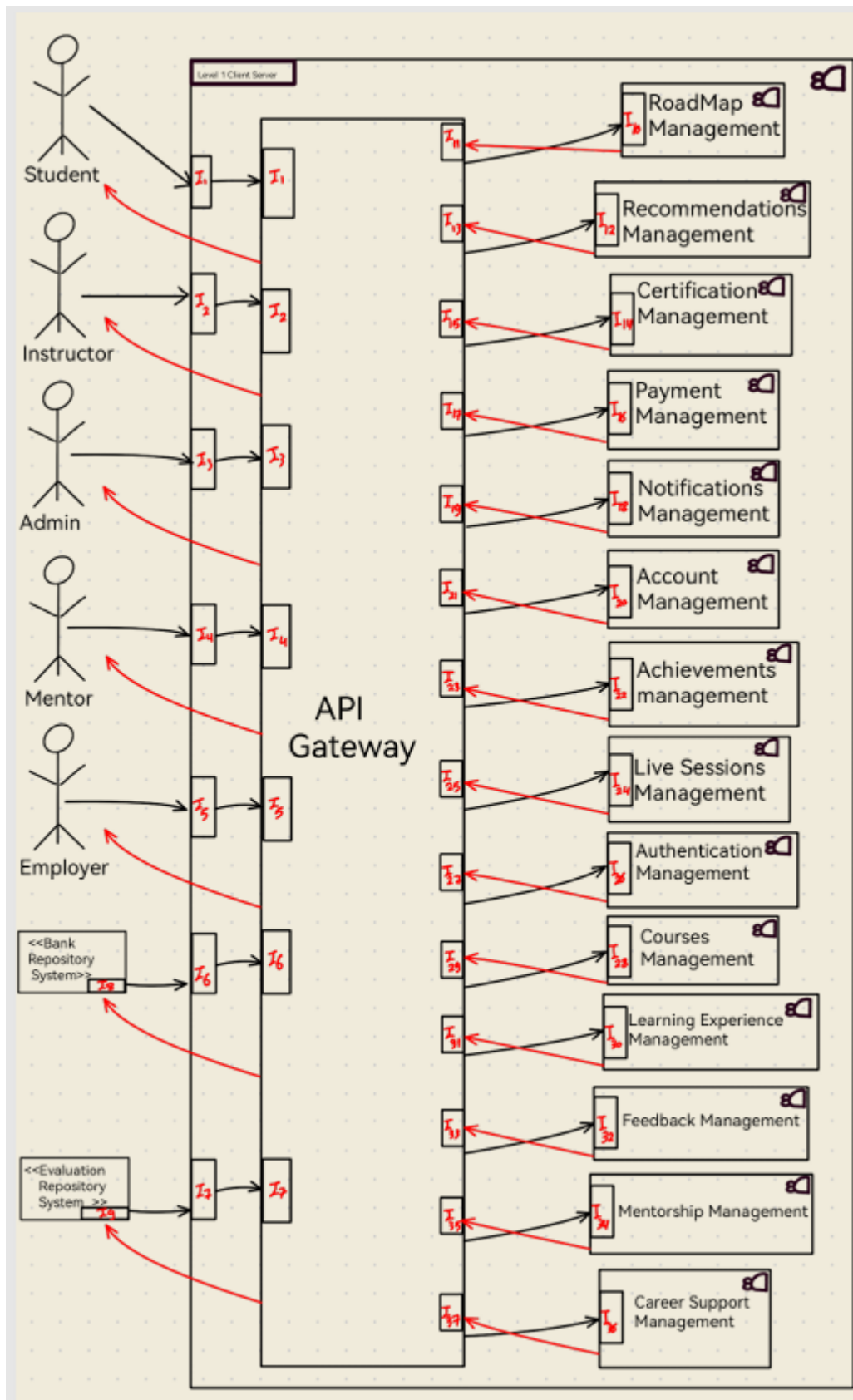
[Link for Code:](#)



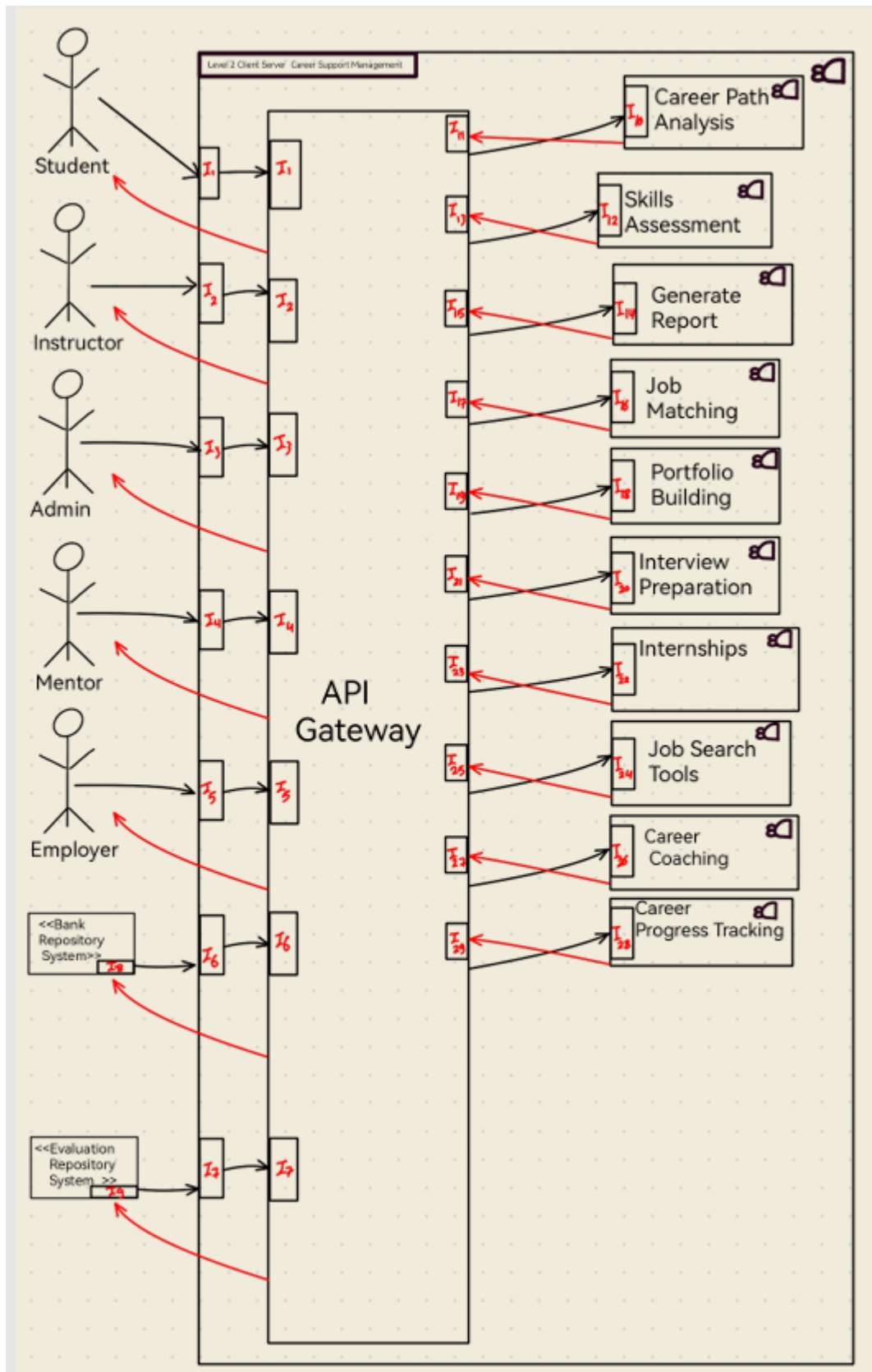
Level 0 Architecture:



Level 1:



Level 2 For Career Support:



Interface Definitions :

External Interfaces (level 0):

Interface I1: Manage Account / Authentication:

```
public interface IAccountAuthenticationManagement {  
  
    // Account operations  
  
    Account createAccount(Account account);  
  
    Account getAccountById(Long id);  
  
    List<Account> getAllAccounts();  
  
    Account updateAccount(Long id, Account updatedAccount);  
  
    void deleteAccount(Long id);  
  
  
    // Authentication operations  
  
    AuthResponse login(LoginRequest request);  
  
    AuthResponse register(RegisterRequest request);  
  
    boolean validateToken(String token);  
  
    void logout(String token);  
  
}
```

Interface I2: Manage Courses / Manage Learning Experience:

```
public interface ICoursesLearningExperienceManagement {  
  
    // Courses  
  
    Course createCourse(Course course);  
  
    Course getCourseById(Long id);  
  
    List<Course> getAllCourses();  
  
    Course updateCourse(Long id, Course updatedCourse);  
  
}
```

```
void deleteCourse(Long id);

// Learning Experience
Experience createExperience(Experience experience);
Experience getExperienceById(Long id);
List<Experience> getAllExperiences();
Experience updateExperience(Long id, Experience updatedExperience);
void deleteExperience(Long id);
}
```

Interface I3: Manage Competitions / Manage Achievements:

```
public interface ICompetitionsAchievementsManagement {

    // Achievements
    Achievement createAchievement(Achievement achievement);
    Achievement getAchievementById(Long id);
    List<Achievement> getAllAchievements();
    Achievement updateAchievement(Long id, Achievement updatedAchievement);
    void deleteAchievement(Long id);


    // Competitions (if needed later)
    Competition createCompetition(Competition competition);
    Competition getCompetitionById(Long id);
    List<Competition> getAllCompetitions();
    Competition updateCompetition(Long id, Competition updatedCompetition);
    void deleteCompetition(Long id);
}
```

Interface I4: Manage Certification / Manage Payments:


```

public interface ICertificationPaymentManagement {

    // Certification

    Certificate issueCertificate(Certificate certificate);

    Certificate getCertificateById(Long id);

    List<Certificate> getAllCertificates();

    Certificate updateCertificate(Long id, Certificate updatedCertificate);

    void revokeCertificate(Long id);


    // Payments

    Payment processPayment(Payment payment);

    Payment getPaymentById(Long id);

    List<Payment> getAllPayments();

    Payment updatePayment(Long id, Payment updatedPayment);

    void cancelPayment(Long id);

}

```

Interface I5: Manage Recommendations / Manage Roadmap:

```

public interface IRecommendationsRoadmapManagement {

    // Recommendations

    Recommendation createRecommendation(Recommendation recommendation);

    Recommendation getRecommendationById(Long id);

    List<Recommendation> getAllRecommendations();

    Recommendation updateRecommendation(Long id, Recommendation
updatedRecommendation);

    void deleteRecommendation(Long id);


    // Roadmap

```

```
RoadMap createRoadMap(RoadMap roadMap);  
RoadMap getRoadMapById(Long id);  
List<RoadMap> getAllRoadMaps();  
RoadMap updateRoadMap(Long id, RoadMap updatedRoadMap);  
void deleteRoadMap(Long id);  
}
```

Interface I6: Manage Mentorship / Manage Career Support:

```
public interface IMentorshipCareerSupportManagement {  
    // Mentorship  
    Mentorship createMentorship(Mentorship mentorship);  
    Mentorship getMentorshipById(Long id);  
    List<Mentorship> getAllMentorships();  
    Mentorship updateMentorship(Long id, Mentorship updatedMentorship);  
    void deleteMentorship(Long id);  
  
    // Career Support  
    CareerSupport createSupport(CareerSupport support);  
    CareerSupport getSupportById(Long id);  
    List<CareerSupport> getAllSupportRecords();  
    CareerSupport updateSupport(Long id, CareerSupport updatedSupport);  
    void deleteSupport(Long id);  
}
```

Interface I7: Manage Notifications:

```
public interface INotificationsManagement {  
    Notification createNotification(Notification notification);  
    Notification getNotificationById(Long id);  
}
```

```
List<Notification> getAllNotifications();

Notification updateNotification(Long id, Notification updatedNotification);

void deleteNotification(Long id);

}
```

Internal Interfaces (level 1):

Interface I8: RoadMap Management:

```
public interface IRoadMapManagement {

    RoadMap createRoadMap(RoadMap roadMap);

    RoadMap getRoadMapById(Long id);

    List<RoadMap> getAllRoadMaps();

    RoadMap updateRoadMap(Long id, RoadMap updatedRoadMap);

    void deleteRoadMap(Long id);

}
```

Interface I9: Recommendations Management:

```
public interface IRecommendationsManagement {

    Recommendation createRecommendation(Recommendation recommendation);

    Recommendation getRecommendationById(Long id);

    List<Recommendation> getAllRecommendations();

    Recommendation updateRecommendation(Long id, Recommendation
updatedRecommendation);

    void deleteRecommendation(Long id);

}
```

Interface I10: Certification Management:

```
public interface ICertificationManagement {
```

```
Certificate issueCertificate(Certificate certificate);  
Certificate getCertificateById(Long id);  
List<Certificate> getAllCertificates();  
Certificate updateCertificate(Long id, Certificate updatedCertificate);  
void revokeCertificate(Long id);  
}
```

Interface I11: Payment Management:

```
public interface IPaymentManagement {  
    Payment processPayment(Payment payment);  
    Payment getPaymentById(Long id);  
    List<Payment> getAllPayments();  
    Payment updatePayment(Long id, Payment updatedPayment);  
    void cancelPayment(Long id);  
}
```

Interface I12: Notifications Management:

```
public interface INotificationsManagement {  
    Notification createNotification(Notification notification);  
    Notification getNotificationById(Long id);  
    List<Notification> getAllNotifications();  
    Notification updateNotification(Long id, Notification updatedNotification);  
    void deleteNotification(Long id);  
}
```

Interface I13: Account Management:

```
public interface IAccountManagement {  
    Account createAccount(Account account);  
    Account getAccountById(Long id);  
    List<Account> getAllAccounts();  
    Account updateAccount(Long id, Account updatedAccount);  
    void deleteAccount(Long id);  
}
```

Interface I14: Achievements Management:

```
public interface IAchievementsManagement {  
    Achievement createAchievement(Achievement achievement);  
    Achievement getAchievementById(Long id);  
    List<Achievement> getAllAchievements();  
    Achievement updateAchievement(Long id, Achievement updatedAchievement);  
    void deleteAchievement(Long id);  
}
```

Interface I15: LiveSessions Management:

```
public interface ILiveSessionsManagement {  
    LiveSession createSession(LiveSession session);  
    LiveSession getSessionById(Long id);  
    List<LiveSession> getAllSessions();  
    LiveSession updateSession(Long id, LiveSession updatedSession);  
    void deleteSession(Long id);  
}
```

Interface I16: Authentication Management:

```
public interface IAuthenticationManagement {  
    AuthResponse login(LoginRequest request);  
    AuthResponse register(RegisterRequest request);  
    boolean validateToken(String token);  
    void logout(String token);  
}
```

Interface I17: Courses Management:

```
public interface ICoursesManagement {  
    Course createCourse(Course course);  
    Course getCourseById(Long id);  
    List<Course> getAllCourses();  
    Course updateCourse(Long id, Course updatedCourse);  
    void deleteCourse(Long id);  
}
```

Interface I18: Learning Experience Management:

```
public interface ILearningExperienceManagement {  
    Experience createExperience(Experience experience);  
    Experience getExperienceById(Long id);  
    List<Experience> getAllExperiences();  
    Experience updateExperience(Long id, Experience updatedExperience);  
    void deleteExperience(Long id);  
}
```

Interface I19: Feedback Management:

```
public interface IFeedbackManagement {  
    Feedback submitFeedback(Feedback feedback);  
    Feedback getFeedbackById(Long id);  
    List<Feedback> getAllFeedbacks();  
    Feedback updateFeedback(Long id, Feedback updatedFeedback);  
    void deleteFeedback(Long id);  
}
```

Interface I20: Mentorship Management:

```
public interface IMentorshipManagement {  
    Mentorship createMentorship(Mentorship mentorship);  
    Mentorship getMentorshipById(Long id);  
    List<Mentorship> getAllMentorships();  
    Mentorship updateMentorship(Long id, Mentorship updatedMentorship);  
    void deleteMentorship(Long id);  
}
```

Interface I21: Career Support Management:

```
public interface ICareerSupportManagement {  
    CareerSupport createSupport(CareerSupport support);  
    CareerSupport getSupportById(Long id);  
    List<CareerSupport> getAllSupportRecords();  
    CareerSupport updateSupport(Long id, CareerSupport updatedSupport);  
    void deleteSupport(Long id);  
}
```

Internal Interfaces (level 2(career support)):

Interface I22: Career Path Analysis Service:

```
public interface ICareerPathAnalysisService {  
  
    CareerPathAnalysis createAnalysis(CareerPathAnalysis analysis);  
  
    CareerPathAnalysis getAnalysisById(Long id);  
  
    List<CareerPathAnalysis> getAllAnalyses();  
  
    CareerPathAnalysis updateAnalysis(Long id, CareerPathAnalysis updatedAnalysis);  
  
    void deleteAnalysis(Long id);  
  
}
```

Interface I23: Skills Assessment Service:

```
public interface ISkillsAssessmentService {  
  
    SkillsAssessment createAssessment(SkillsAssessment assessment);  
  
    SkillsAssessment getAssessmentById(Long id);  
  
    List<SkillsAssessment> getAllAssessments();  
  
    SkillsAssessment updateAssessment(Long id, SkillsAssessment updatedAssessment);  
  
    void deleteAssessment(Long id);  
  
}
```

Interface I24: Report Generation Service:

```
public interface IReportGenerationService {  
  
    Report generateReport(ReportRequest request);  
  
    Report getReportById(Long id);  
  
    List<Report> getAllReports();  
  
    Report updateReport(Long id, Report updatedReport);  
  
}
```



```
void deleteReport(Long id);  
}
```

Interface I25: Job Matching Service:

```
public interface IJobMatchingService {  
    JobMatch createJobMatch(JobMatch match);  
    JobMatch getJobMatchById(Long id);  
    List<JobMatch> getAllJobMatches();  
    JobMatch updateJobMatch(Long id, JobMatch updatedMatch);  
    void deleteJobMatch(Long id);  
}
```

Interface I26: Portfolio Service:

```
public interface IPortfolioService {  
    Portfolio createPortfolio(Portfolio portfolio);  
    Portfolio getPortfolioById(Long id);  
    List<Portfolio> getAllPortfolios();  
    Portfolio updatePortfolio(Long id, Portfolio updatedPortfolio);  
    void deletePortfolio(Long id);  
}
```

Interface I27: Interview Preparation Service:

```
public interface IInterviewPreparationService {  
    InterviewSession createSession(InterviewSession session);  
    InterviewSession getSessionById(Long id);  
    List<InterviewSession> getAllSessions();  
}
```

```
InterviewSession updateSession(Long id, InterviewSession updatedSession);  
void deleteSession(Long id);  
}
```

Interface I28: Intern ship Service:

```
public interface IInternshipService {  
    Internship createInternship(Internship internship);  
    Internship getInternshipById(Long id);  
    List<Internship> getAllInternships();  
    Internship updateInternship(Long id, Internship updatedInternship);  
    void deleteInternship(Long id);  
}
```

Interface I29: Job Search Service:

```
public interface IJobSearchService {  
    JobSearchRecord createSearchRecord(JobSearchRecord record);  
    JobSearchRecord getSearchRecordById(Long id);  
    List<JobSearchRecord> getAllSearchRecords();  
    JobSearchRecord updateSearchRecord(Long id, JobSearchRecord updatedRecord);  
    void deleteSearchRecord(Long id);  
}
```

Interface I30: Career Coaching Service:

```
public interface ICareerCoachingService {  
    CoachingSession createSession(CoachingSession session);  
    CoachingSession getSessionById(Long id);  
}
```

```

List<CoachingSession> getAllSessions();

CoachingSession updateSession(Long id, CoachingSession updatedSession);

void deleteSession(Long id);

}

```

Interface I31: Career Progress Tracking Service:

```

public interface ICareerProgressTrackingService {

    ProgressRecord createRecord(ProgressRecord record);

    ProgressRecord getRecordById(Long id);

    List<ProgressRecord> getAllRecords();

    ProgressRecord updateRecord(Long id, ProgressRecord updatedRecord);

    void deleteRecord(Long id);

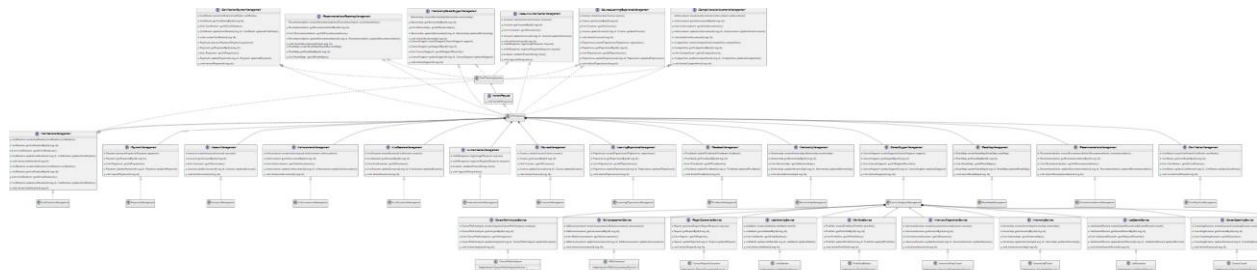
}

```

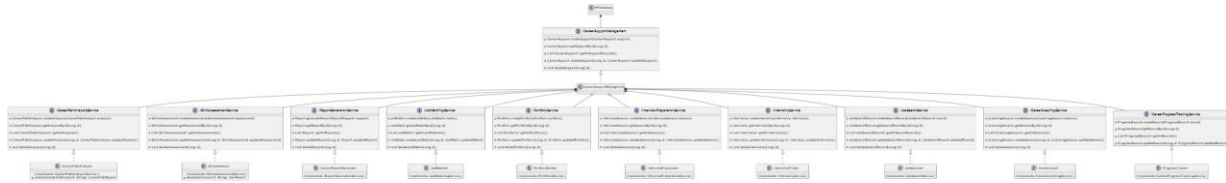
Class Diagram:

[Link For All Code:](#)

[Link For Level2:](#)



Level2:



Traceability matrix :

Link:

Requirement	Auth Management	AuthZ Management	Payment Processing	Course Mgmt	Learning Experience	Live Learning	Project Assignment	Course Recommendation	Certification Mgmt	Career Development	Mentorship Mgmt	Competency Mgmt	Learning Pathway	Platform Admin	Content Security	Test Case IDs	Status
1. Secure login (MFA/SSO/2FA)	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	TC_1, TC_2, TC_3	NAH
2. RBAC	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	TC_7	NAH
3. PayPal/Stripe Integration	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	TC_19-TC_27	NAH
4. AI course recommendations	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	TC_4	NAH
5. GDPR/CCPA compliance	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	TC_8, TC_11	NAH
6. Real-time progress tracking	✗	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	TC_2, TC_4	NAH
7. AI-generated content	✗	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	TC_4	NAH
8. Blockchain certifications	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	TC_6	NAH
9. Badges/Achievements	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	TC_12	NAH
10. Resume builder/job matching	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	TC_2, TC_3	NAH
11. GDPR/Data privacy	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	TC_11	NAH
12. Admin dashboard	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	TC_7	NAH
13. Subscription tiers	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	TC_19-TC_27	NAH
14. Adaptive learning paths	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✓	✗	✗	TC_1	NAH
15. Multi-format content	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	TC_4	NAH

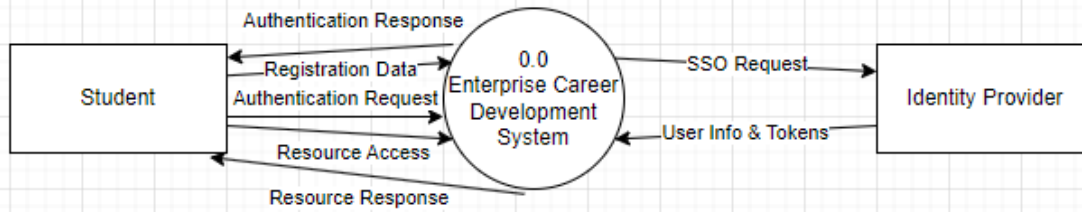
Sequence & Data Flow Diagrams

1- Manage Authentication

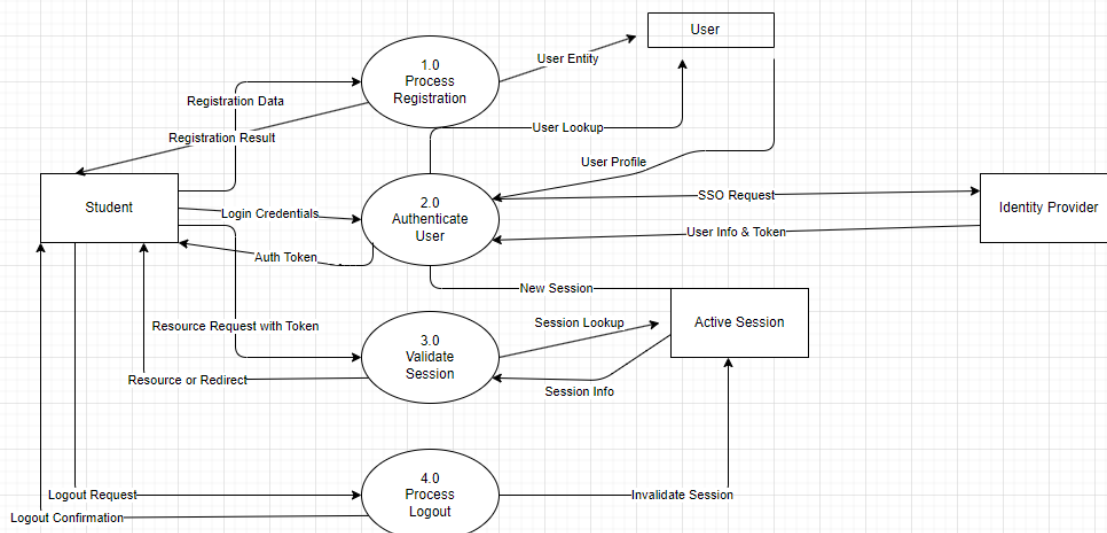
[Link For Image:](#)

[Link For Code:](#)

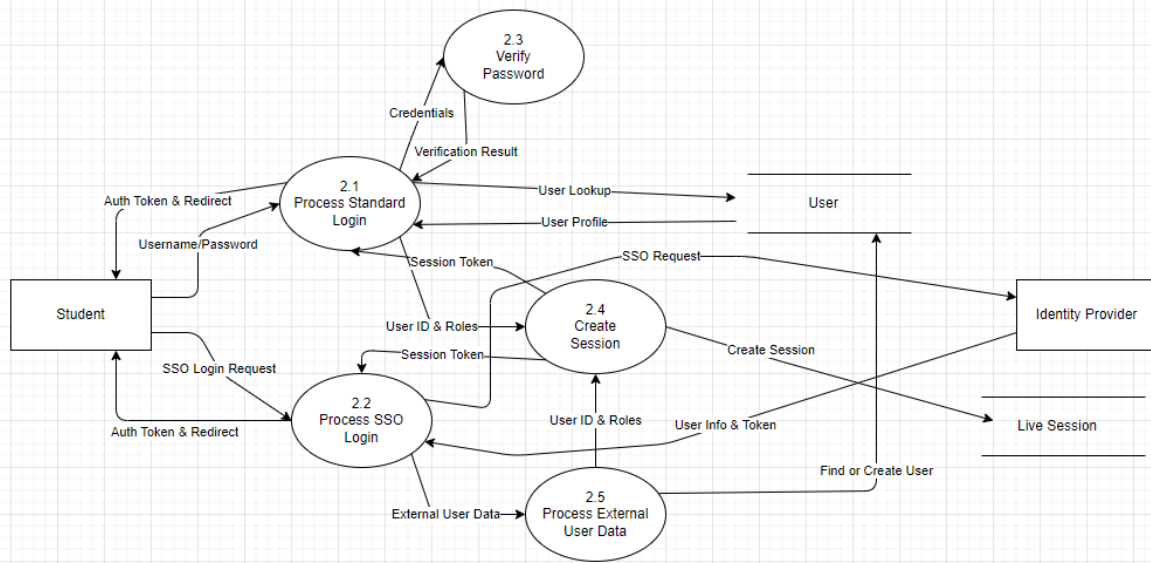
Manage Authentication - Context Diagram (Level 0)



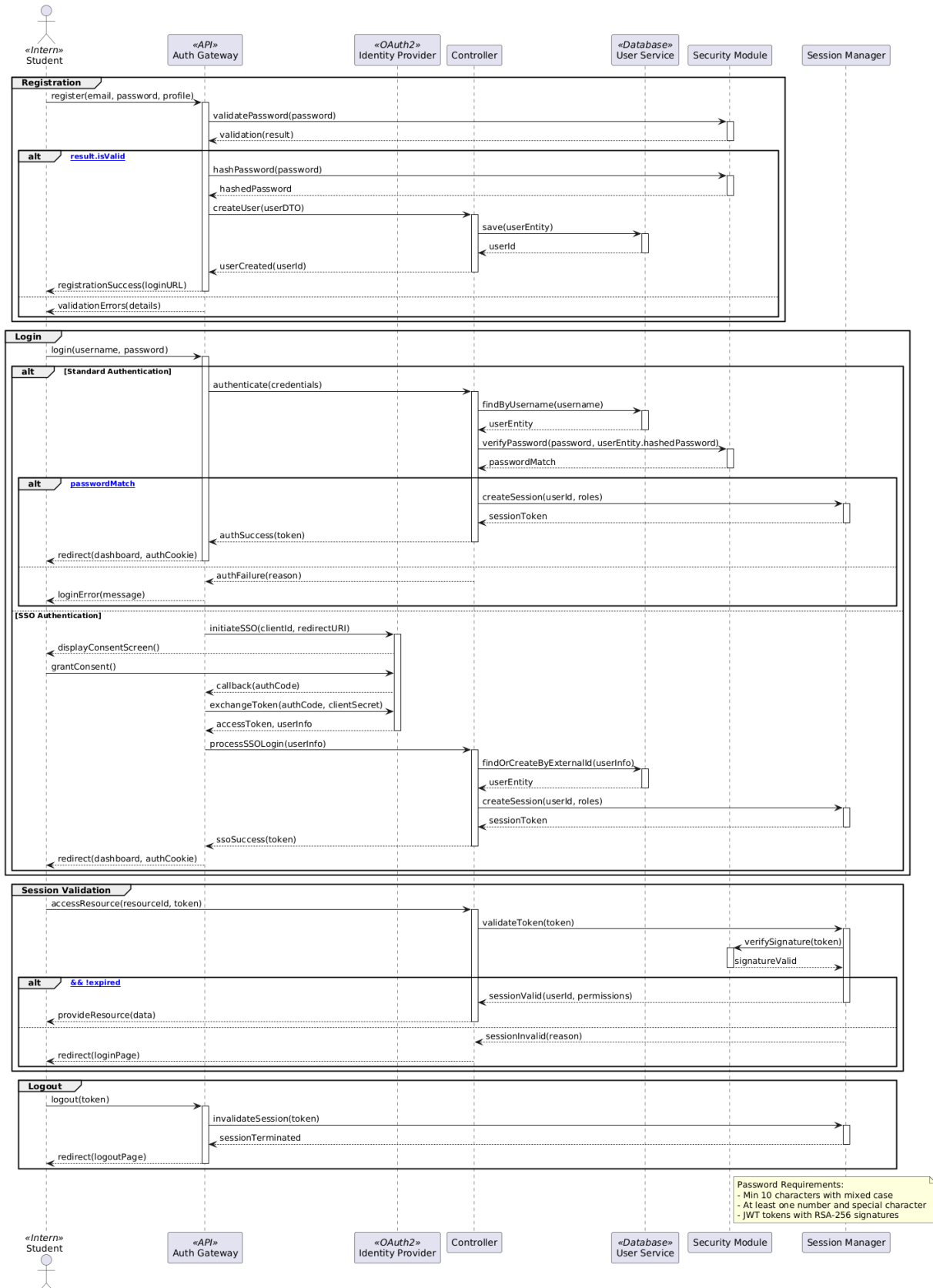
Manage Authentication - Level 1 DFD



Manage Authentication - Level 2 DFD (Authentication Process)



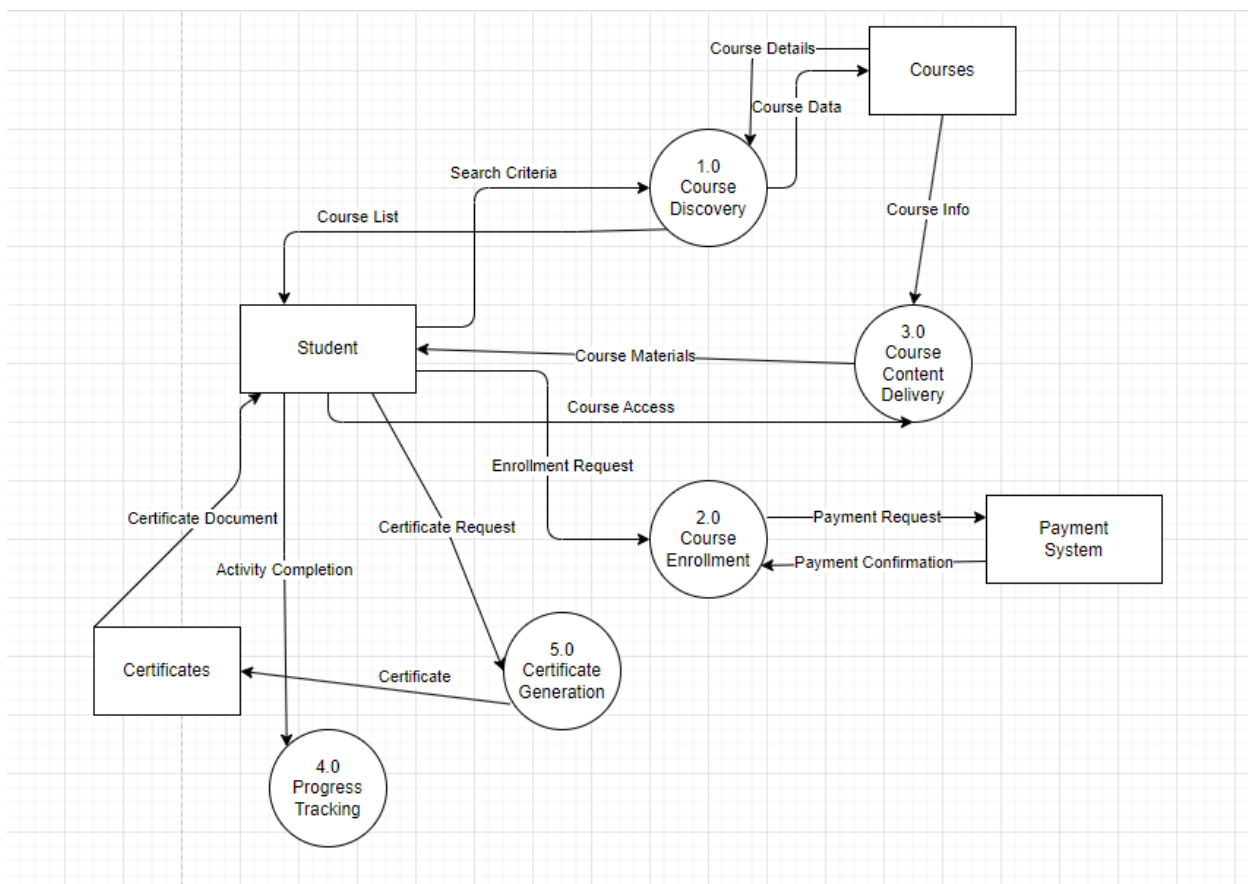
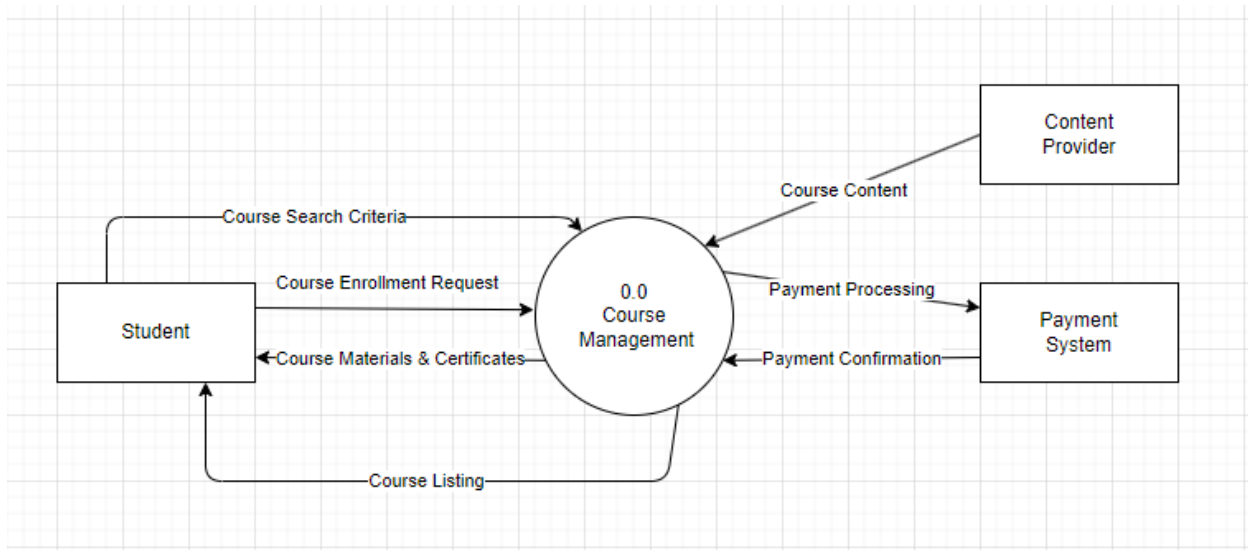
Enterprise Career Development System
Authentication | v3.2.1 | SEQUENCE DIAGRAM

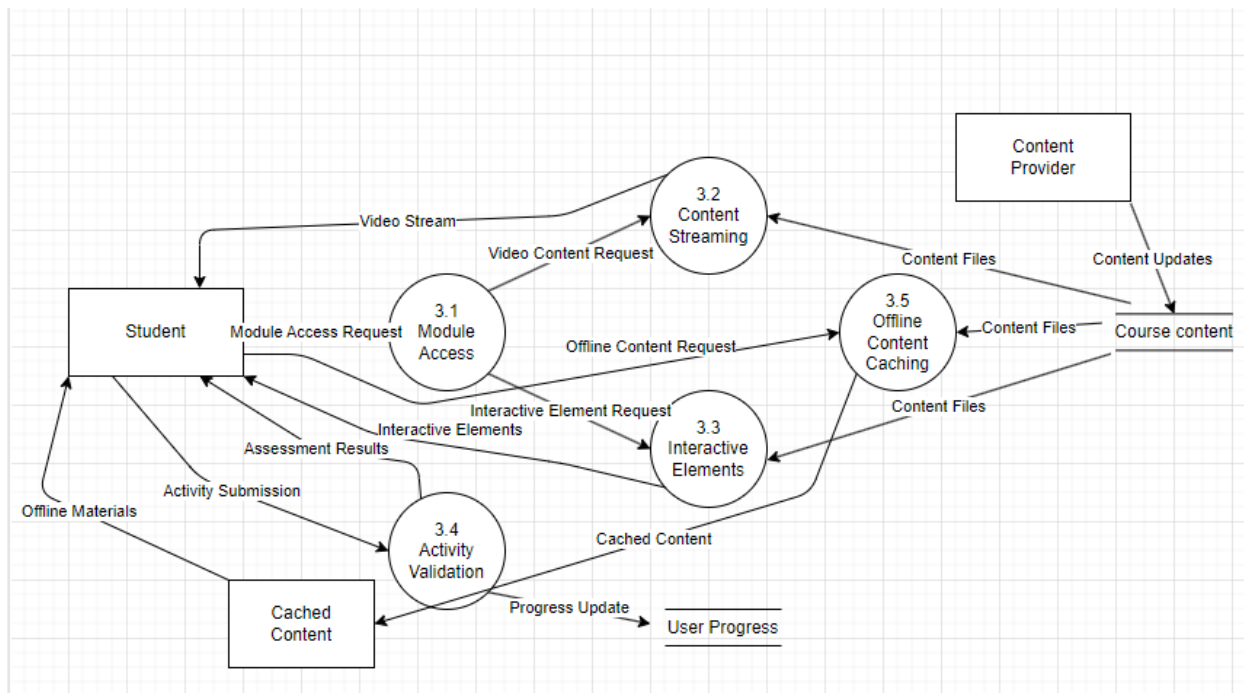


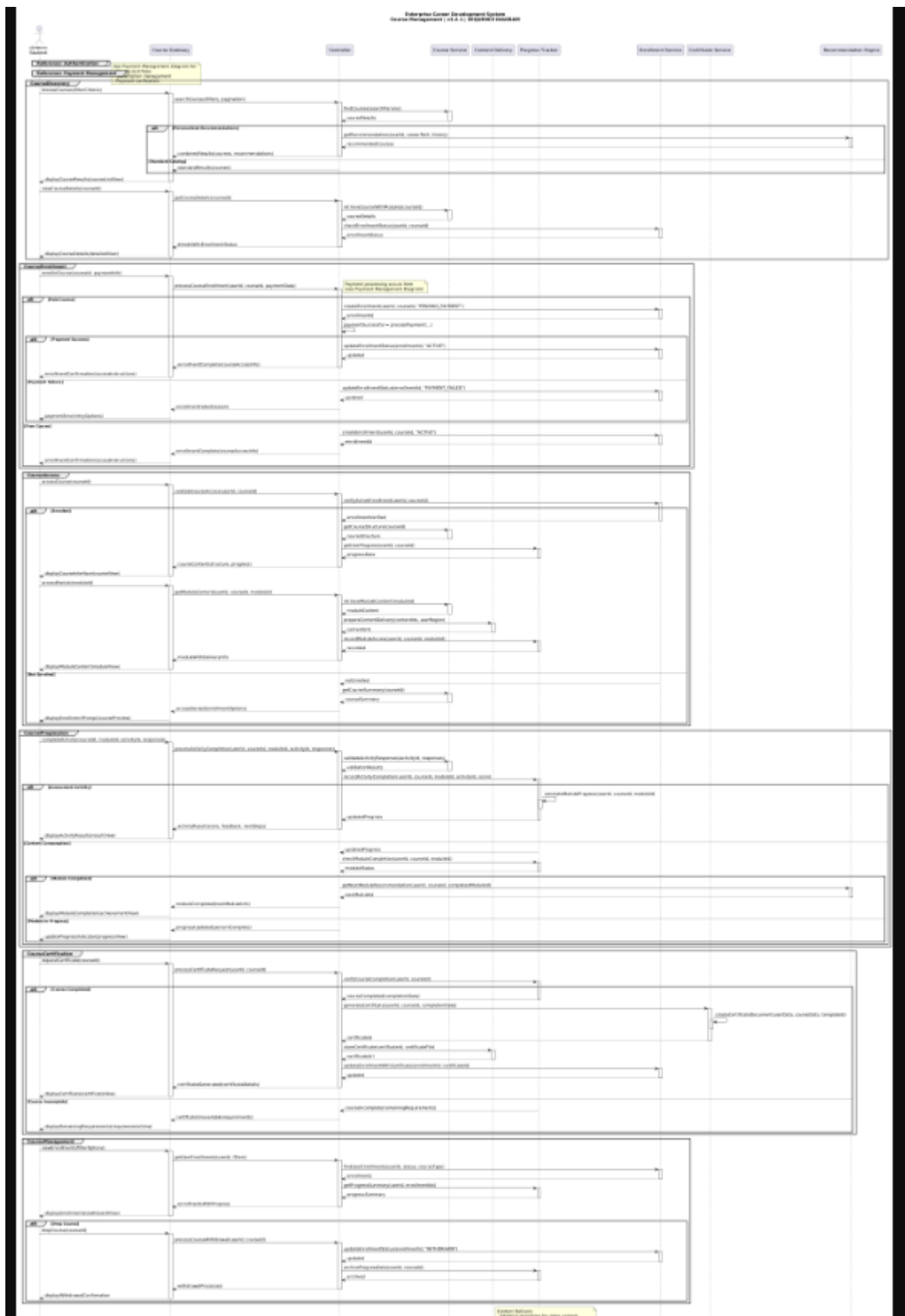
2-Manage Courses

[Link For Image:](#)

[Link For Code:](#)



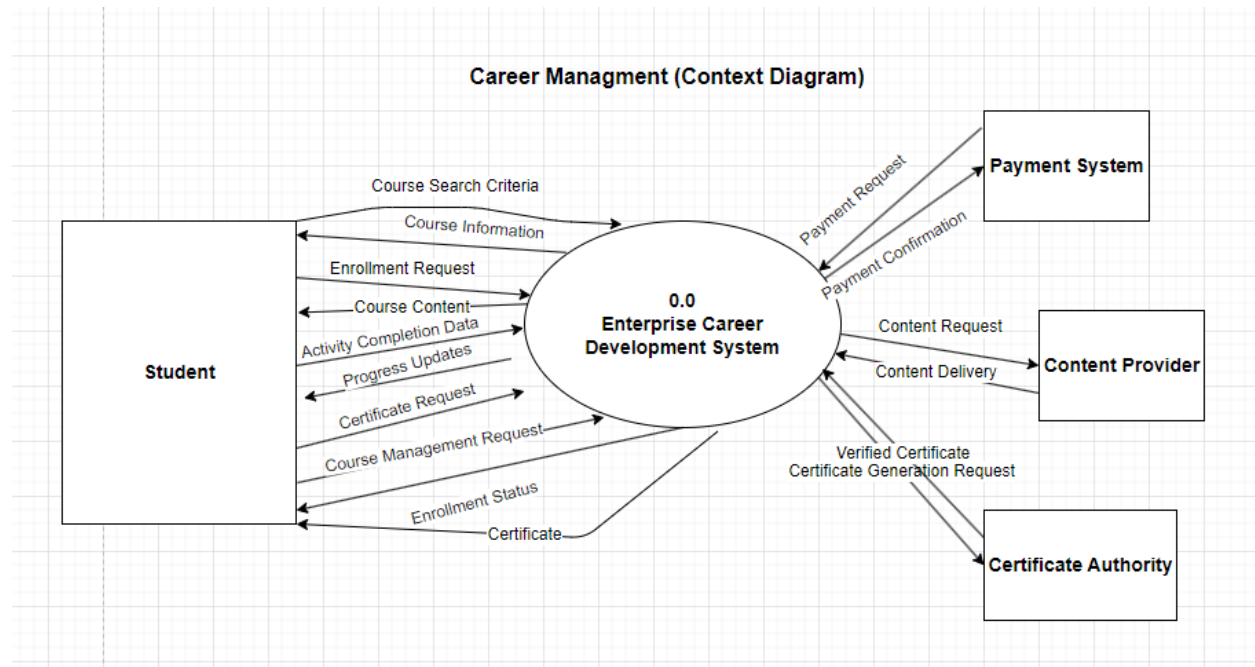


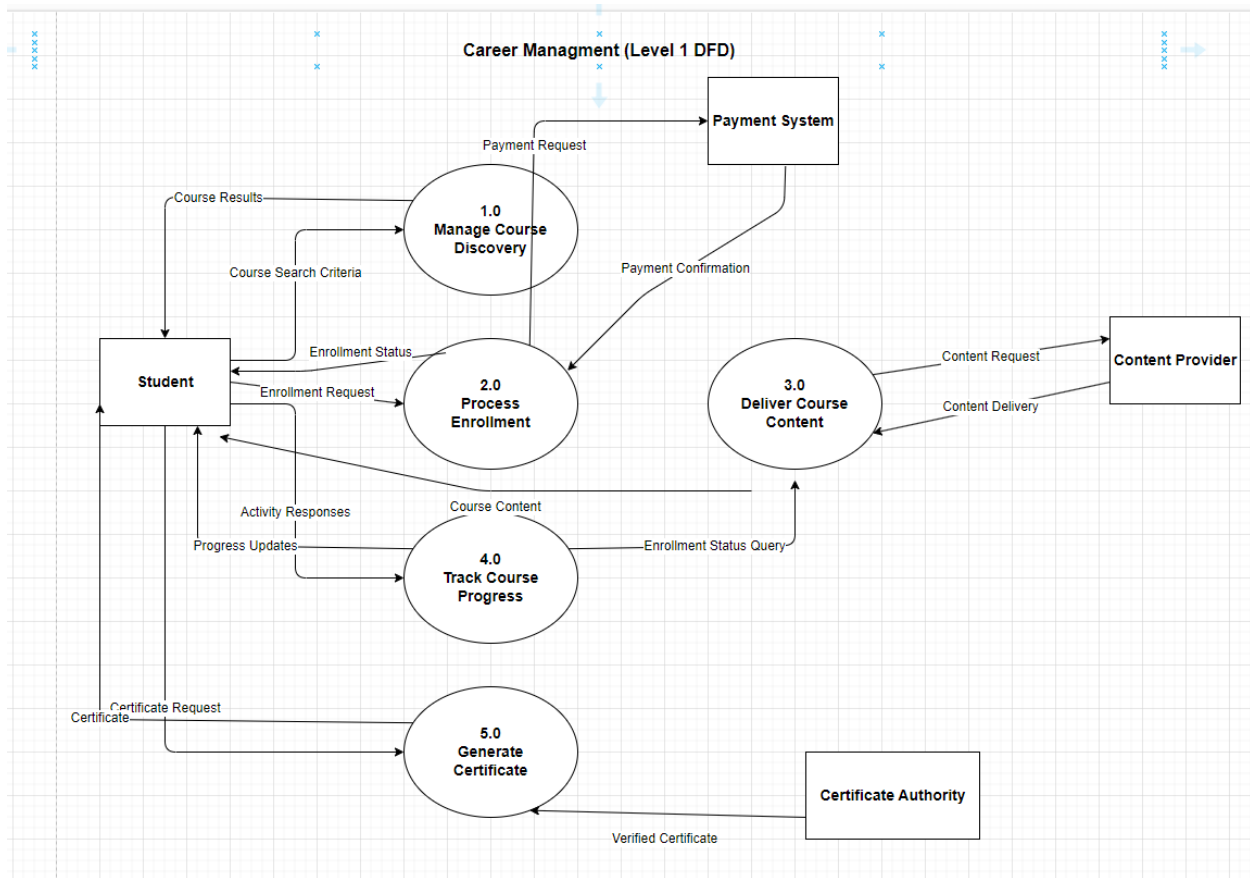


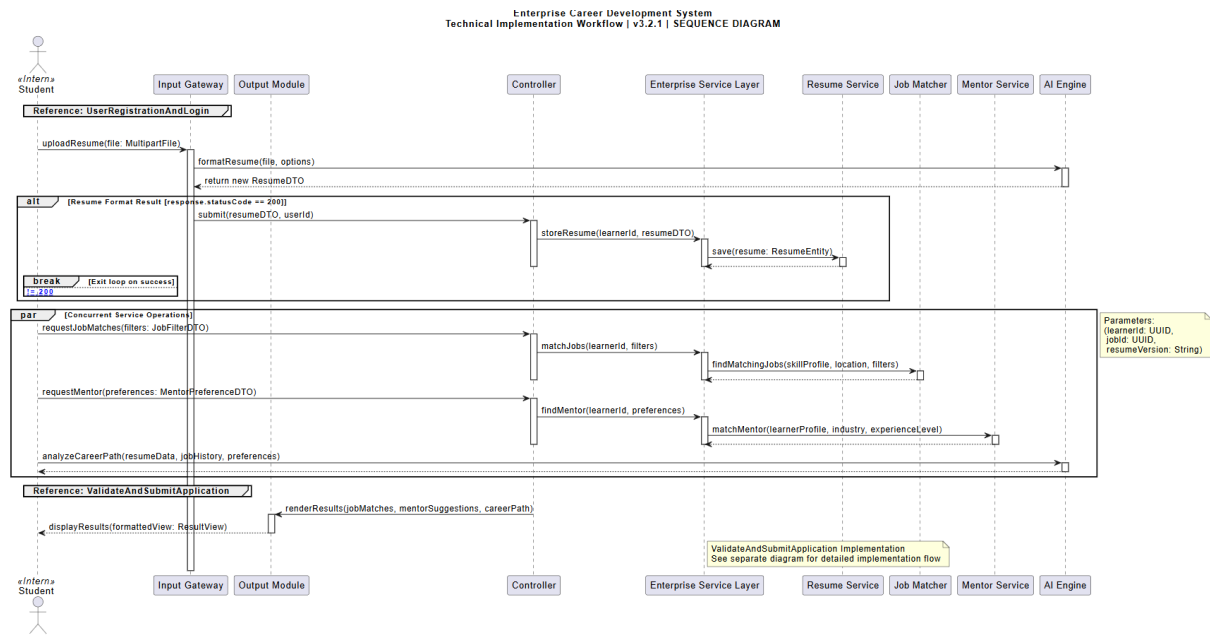
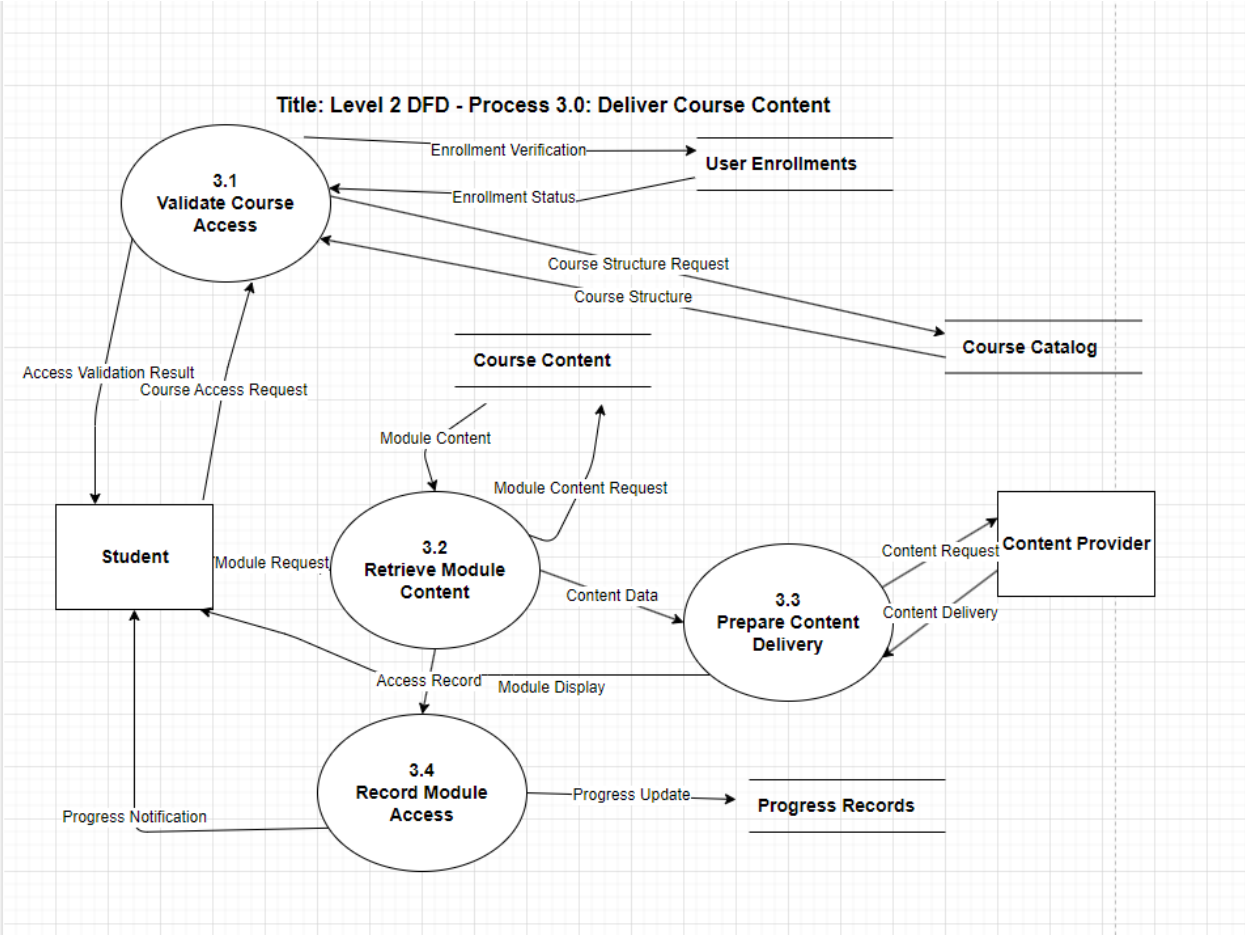
3- Career Management:

[Link For Code:](#)

[Link For Image:](#)



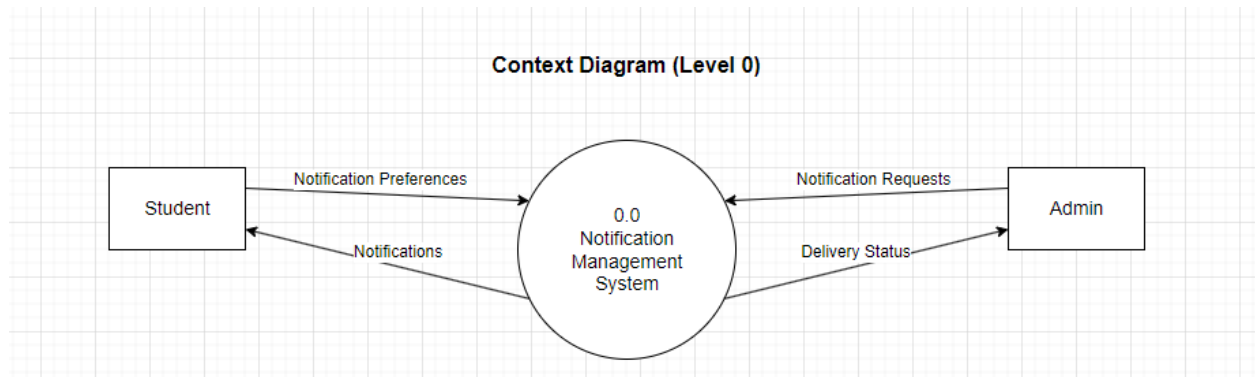


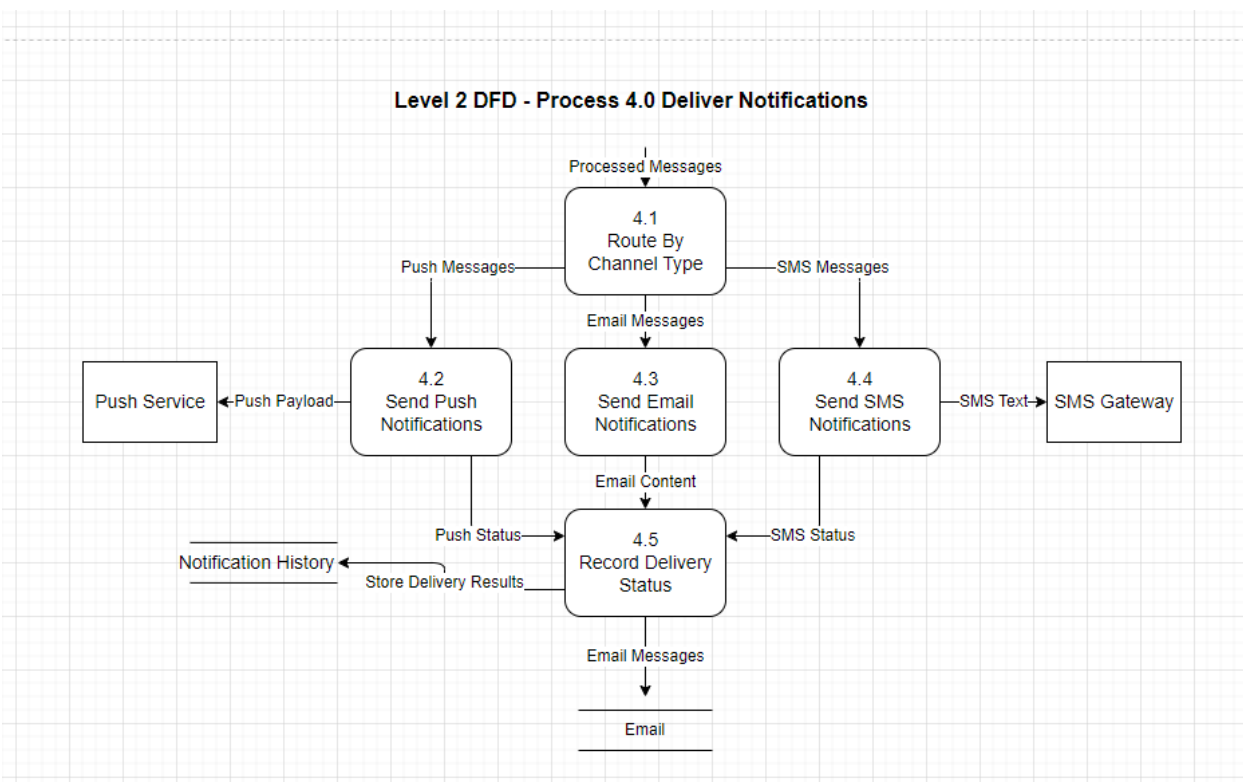
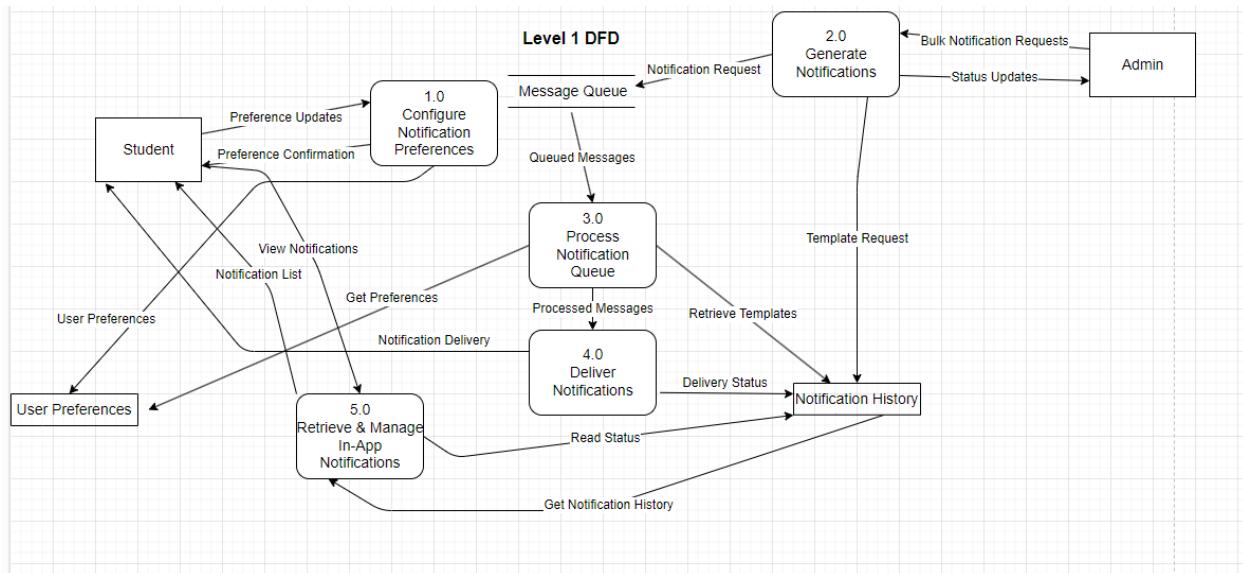


4- Notification Management:

[Link For Image:](#)

[Link For Code:](#)

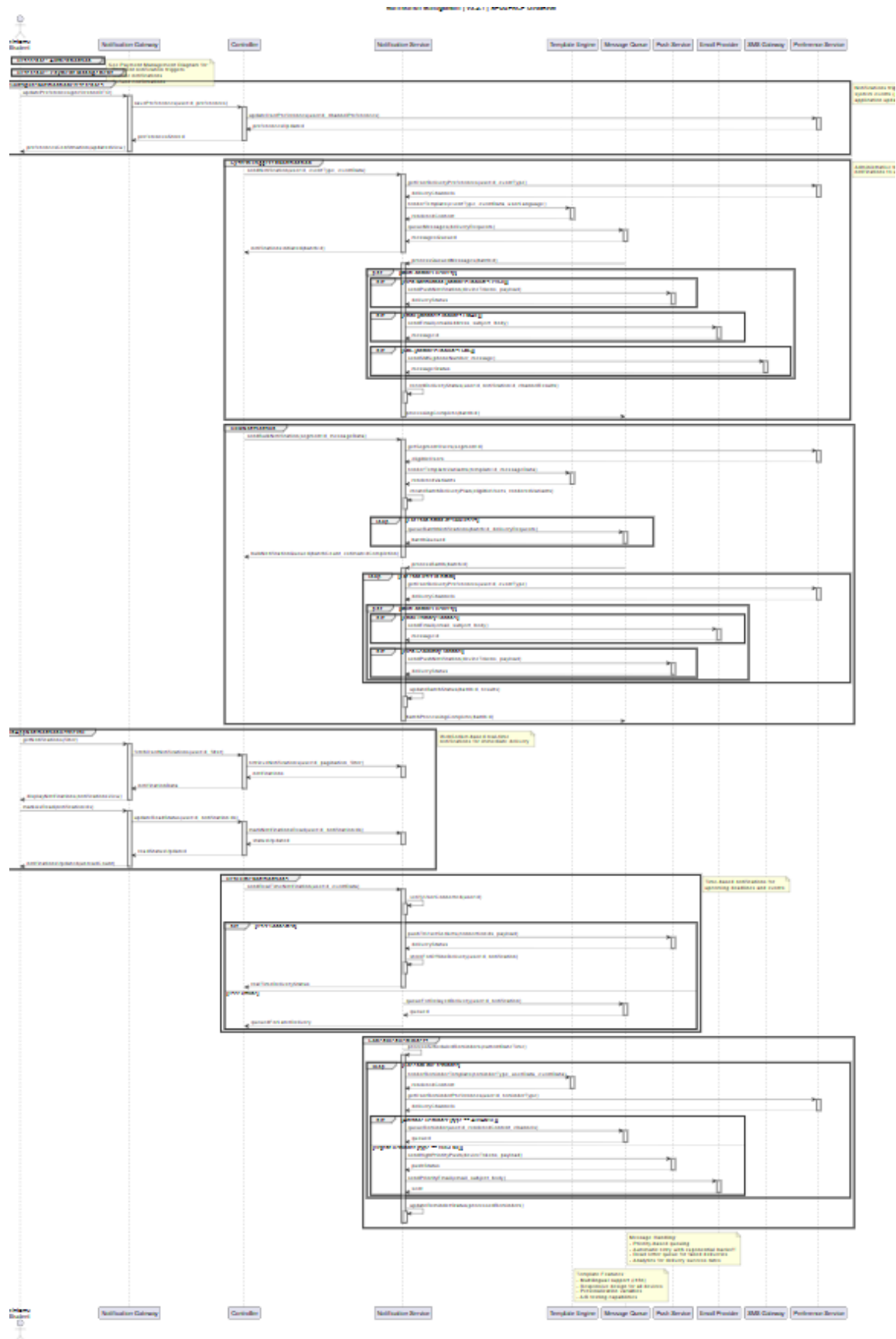




5- Management Payment:

[Link For Image:](#)

[Link For Code:](#)



Task8:

1. Use Case Estimations (Fibonacci Points)

Use Case	Estimation (Fibonacci Points)
Manage Authentication	8
Manage User Roles	5
Manage Payments	13
Manage Courses	13
Manage Learning Experience	8
Manage AI-Powered Learning	5
Manage Achievements	3
Manage Live Sessions	8

Manage Career Support	5
Manage Mentorship	5
Manage Recommendations	3
Manage Certification	5
Manage Competitions	3
Manage RoadMap	5
Manage Platform	8
Manage Security	8
Manage Notifications	3
Manage Account	5
Manage Feedback	2
Manage Subscription	3

2.Gantt Chart

Phase 1: Core Functionality (Weeks 1-10)

Use Case	Duration (Weeks)	Start Week	End Week
Manage Authentication	8	1	8
Manage User Roles	5	3	7
Manage Payments	13	1	13
Manage Courses	13	2	14
Manage Learning Experience	8	5	12

Phase 2: Advanced Features (Weeks 11-20)

Use Case	Duration (Weeks)	Start Week	End Week
Manage AI-Powered Learning	5	11	15

Manage Live Sessions	8	12	19
Manage Career Support	5	14	18
Manage Mentorship	5	15	19
Manage Certification	5	16	20

Phase 3: Enhancements & Finalization (Weeks 21-25)

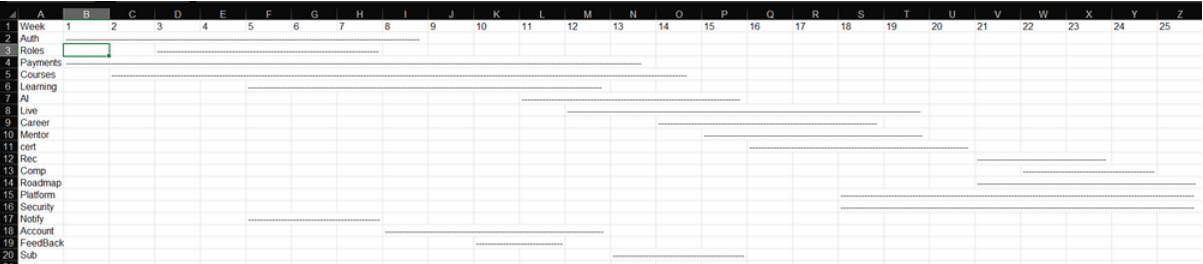
Use Case	Duration (Weeks)	Start Week	End Week
Manage Recommendations	3	21	23
Manage Competitions	3	22	24
Manage RoadMap	5	21	25

Manage Platform	8	18	25
Manage Security	8	18	25

Phase 4: Minor Use Cases (Parallel)

Use Case	Duration (Weeks)	Start Week	End Week
Manage Notifications	3	5	7
Manage Account	5	8	12
Manage Feedback	2	10	11
Manage Subscription	3	13	15

Visual Gantt Chart (Simplified Representation):



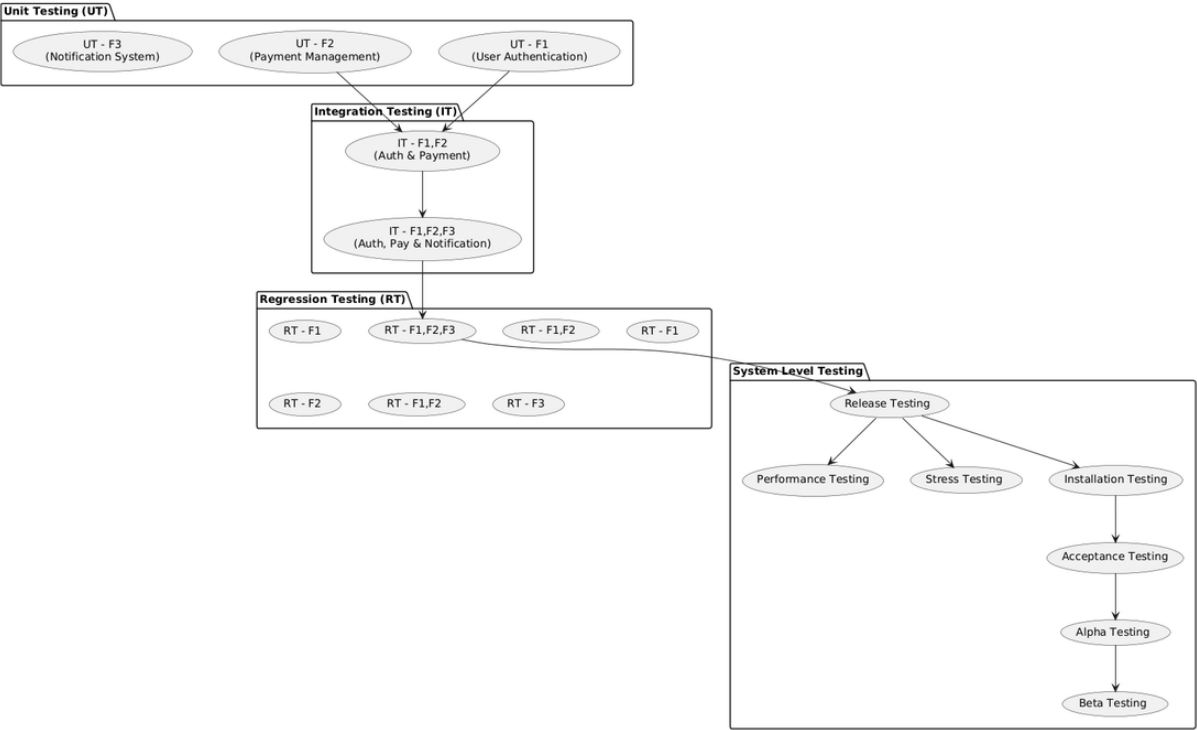
[Link](#)

Task 9 : Test Plan

Test Name	Main Features Involved
1. Unit Testing (UT) - F1	Functionality 1 (e.g., User Authentication)
2. Unit Testing (UT) - F2	Functionality 2 (e.g., Payment Management)
3. Unit Testing (UT) - F3	Functionality 3 (e.g., Notification System)
4. Integration Testing (IT)	Combined F1 & F2
5. Regression Testing (RT)	Feature F1
6. Regression Testing (RT)	Features F1 & F2
7. Integration Testing (IT)	Features F1, F2 & F3
8. Regression Testing (RT)	Feature F1
9. Regression Testing (RT)	Feature F2
10. Regression Testing (RT)	Features F1 & F2

11. Regression Testing (RT)	Feature F3
12. Regression Testing (RT)	Features F1, F2 & F3
13. Release Testing	Full system readiness
14. Performance Testing	System responsiveness and throughput
15. Stress Testing	System behavior under peak workloads
16. Installation Testing	Deployment and installation verification
17. Acceptance Testing	User acceptance and validation
18. Alpha Testing	Early internal testing phase
19. Beta Testing	User group testing prior to official release

Testing Plan for the Project



[Link](#)